June 16th,2024

Feedback Control System Assignment Report

Embedded System 3, LAB 06

Document Version 1.0

Edited by:
Farros Ramzy (3767353)

Abstract

This technical report describes about the use of PID, how to calculate them, and how to implement them in the robotic system which required a movement output to make it move smoothly from one point to another. (And to be known, this assignment is started by Farros Ramzy at the same time on the ES3 robot project. Every part that was written here might also already be implemented on the robot, except the calculation on the MATLAB Simulink.)

List of Figures

List of Tables

Table of Contents

1. Introduction

PID stands for Proportional-Integral-Derivative, which is a type of feedback control system commonly used in industrial control systems to maintain a desired output level.

The PID controller meant to continuously calculates the error value and applies corrections based on these three terms to ensure the process variable reaches and maintains the desired setpoint.

This assignment is about calculating the proper PID values and making a pseudo code or skeleton function which will be implemented later for the robot project.

Must be known, every moving object has a start point and the end point. An to arrive at the end point, the object must detect it surroundings. While the unknown behaviors appeared during the movement, a feedback control system is needed to control the and maintain the object performance, so it does not overshoot or missed the target point.

Each of the PID can be elaborated as these explanations below.

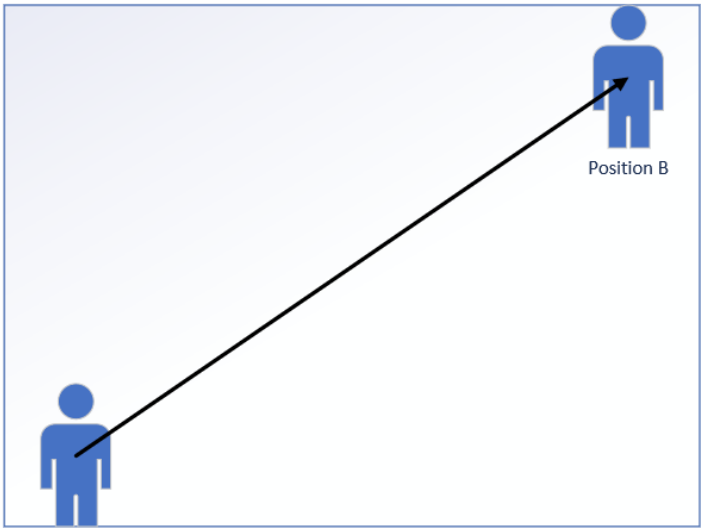

Figure 1. 1 A Proportional (P) concept.

This Figure 1. 1 on the left is a representation of the proportional. Proportional is the distance between the start point to the target. As the object move from position A to B, the distance between the object to its position will become shortened. This distance change is a value needed for the integral of the PID to control the change of velocity of the object as it is moving to the target.

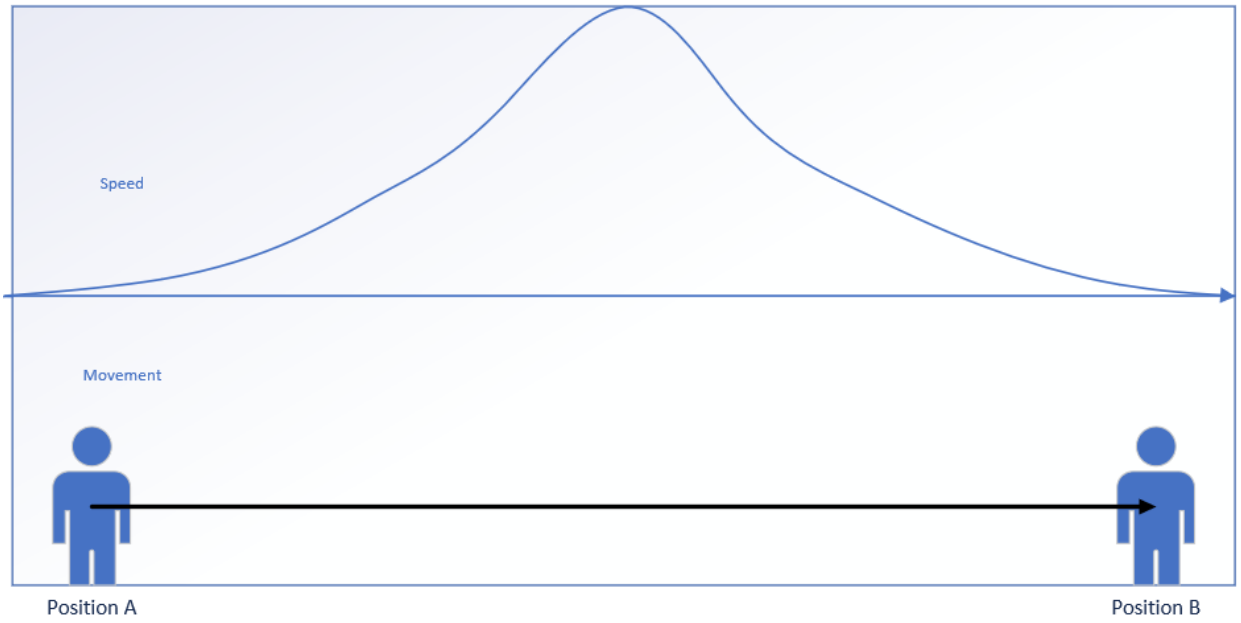

Figure 1. 2 An Integral (I) concept.

This Figure 1. 2 above is a representation of an integral, which is an accumulation to speed up and to slow down the velocity of an object due to its proportional distance to the set point.

The set point here is depicted as the Position B.

As the object start moving from Position A, the integral will speed up the object until it become close enough to the setpoint, then the integral will slow down the moving system until it stopped, exactly on the set point.

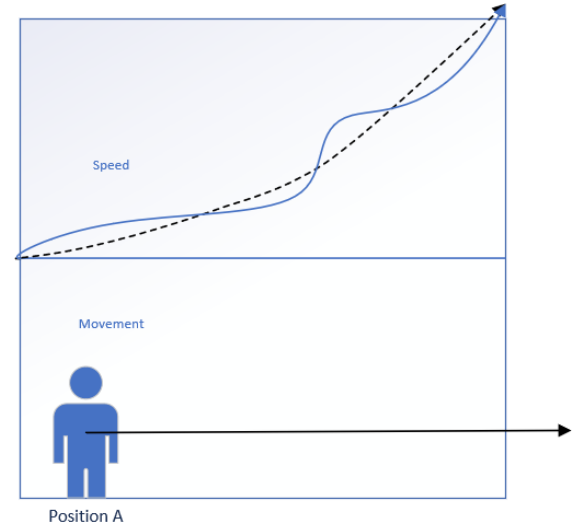

Figure 1. 3 A Derivative (D) concept.

Then, this Figure 1. 3 on the left represents the derivative behavior of the PID. Derivative is an accumulation to control the integral while changing the speed of the object to prevent an oscillation, considering the quick-change behavior of the integral itself. The derivative task is to make sure that the change of the object's velocity is always aligned with the ideal change, which is drawn as the dotted curve line in this picture on the left.

After understanding the concept of the PID, some procedures can be done for the FCS assignment.

1.1. Equipment

The equipment needed to do this research practicum are:

1. A programming platform for the STM32 board (STM32CubeIDE)
2. A Simulink App From MATLAB

2. Procedure

2.1. Calculating PID

To start on the PID feedback control system, some calculations are needed to be done. These calculations can be done manually or by using the PID Simulink app in MATLAB.

And since the assignment is very important for the upcoming robot project, let us just do both calculation methods for the parallax servo to prepare the robot.

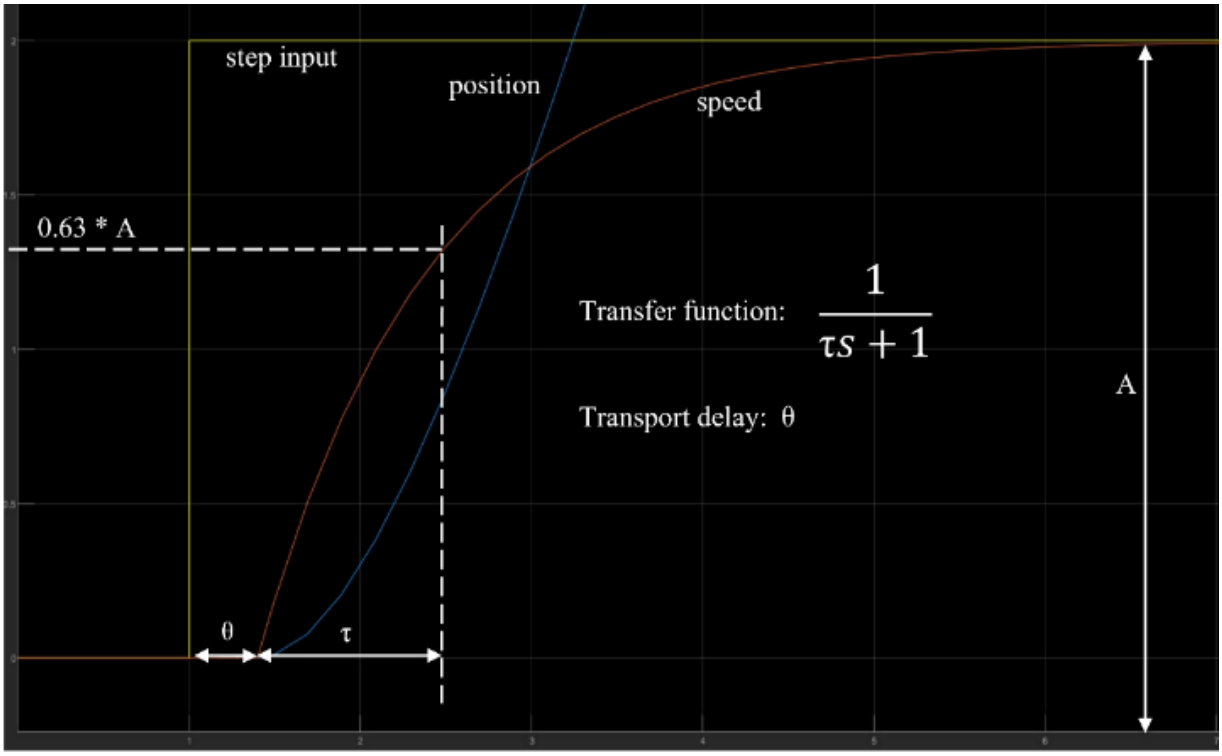

Figure 2. 1. 1 A capture concept of the position-control formula.

This Figure 2. 1. 1 above is a concept of how to get the PID value for the robot system. This method contains the default multiplication of a servo speed area, the formula of the transport function, and a hint of where the transport delay and transport change are placed.

This information will be useful to do the manual calculation of the PID by using the Ziegler-Nichols method.

To simulate the PID in the Simulink MATLAB, a design must be created. And this Figure 2. 1. 2 below is the simulation design of the servo using PID method in Simulink MATLAB.

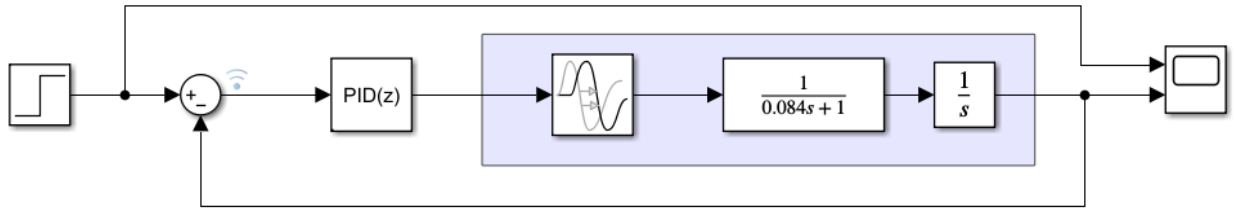

Figure 2. 1. 2 A PID simulation design from Simulink.

By using this simulation design, a method to find the PID values for the robot can be easier than by only using the manual calculations.

After finishing the design, calculations for the parallax servo system can be done by using the Ziegler-Nichols method. And just for a reminder, adding a simulation from this manual calculation in Simulink from the MATLAB (as an example) would improve the feedback control's precision than just by holding on the manual calculation.

To start with the calculations, a reference to the parallax rotation speed data must be read first. This data provides the average speed and the starting point of the servo in two conditions, which are the loaded and unloaded servo.

And because this calculation will be used for a robot which must have two wheel's controllers that also holds a weight of its body, it is better to use the loaded version.

And by this decision, a data taken from the parallax speed datasheet can be listed in this Table 2. 1. 1 below.

Table 2. 1. 1 Servo Parallax speed and position data.

Name	Timestamp (ms)	Change of Position (Δ pos)	Servo position	Speed load (A)
Start point	41	0	0	0.05
Right wheel	819	2	1436	1.95
Left wheel	884	2	1571	2

From this data and the graph of the formula in the simulation concept above, we can do some manual calculations to get the kP, kI, and kD constants for the PID controller of the robot.

As shown in the graph of the formula in the design, speed of servo can be searched by:

$S = 0.63 \times A$

And the period (T) from the θ to τ is formulated in:

$T = \tau + \theta$

From these, the kP, kI, and kD calculations for the left and right wheels' PID can be listed as below:

Left Wheel:

- Speed

  $S_r = 0.63 \times A_r$
  $S_r = 0.63 \times 2$
  $\boldsymbol{S_r = 1.26}$
  (This value is precisely equal to the speed with Δ 2 pos in the parallax table)
- Period

  $T_r = \tau_r + \theta$
  T_r to the S_r with 1.26 value in the parallax data document is 127
  $T_r = \tau_r + \theta$
  127 = $\tau_r + 41$
  $\tau_r = 127 - 41$
  $\tau_r = 86\ ms$
  $\boldsymbol{\tau_r = 0.086\ s}$

Right Wheel:

- Speed

  $S_r = 0.63 \times A_r$
  $S_r = 0.63 \times 1.95$
  $\boldsymbol{S_r = 1.228}$
  (This value close by the 1.21 speed value with Δ 2 pos in the parallax table)
- Period

  $T_r = \tau_r + \theta$
  T_r to the S_r with 1.21 value in the parallax data document is 125
  $T_r = \tau_r + \theta$
  125 = $\tau_r + 41$
  $\tau_r = 125 - 41$
  $\tau_r = 84\ ms$
  $\boldsymbol{\tau_r = 0.084\ s}$

After calculating both necessary speed and period, now is the time to use the Simulink App from the MATLAB to find the maximum oscillation. This oscillation can be searched by adding the transport delay and the period to the transfer function to the PID design in Simulink.

Then, simulate the scope of that design multiple times, and changing only the proportional value to get the balanced volatile graphic which is presented in this Figure 2. 1. 3 below.

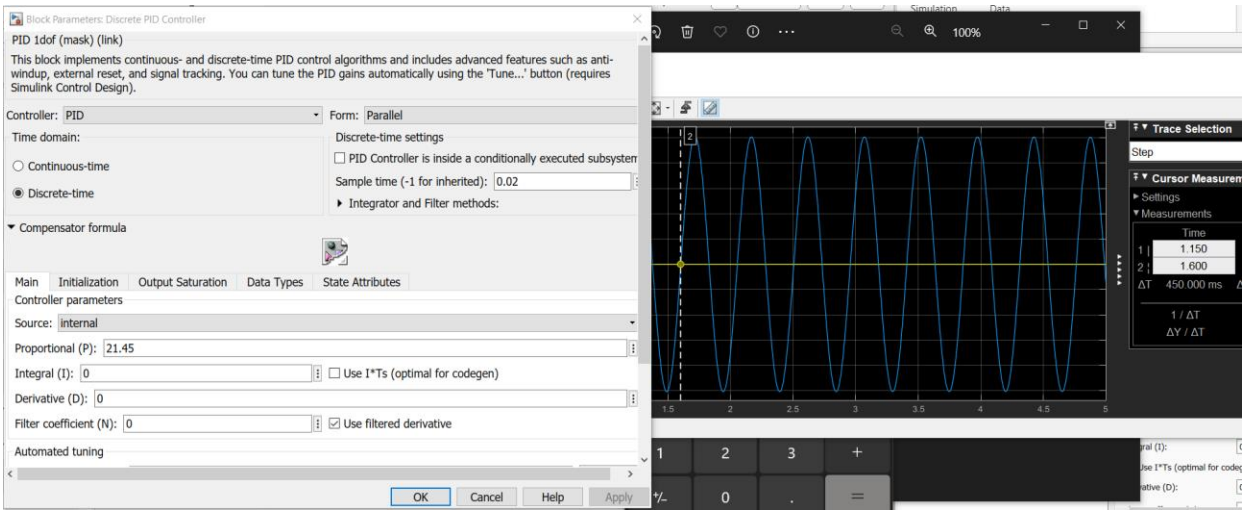

Figure 2. 1. 3 PID mask graph on maximum oscillation.

This volatile graphic on Figure 2. 1. 3 above is the image of the maximum oscillation of the servo. By this, it is known that the max oscillation of the right wheel is about 21.45. and the left wheel is about 21.415.

The time change between one wave in this oscillation (ΔT_u) of the right wheel is 0.45 ms. And the time change between one wave in this oscillation (ΔT_u) of the left wheel is 0.458 ms.

From this value and the calculation using the Ziegler-Nichols method, the PID that were received are:

kP = 5.2697

kI = 2.413

kD = 0.3339

However, these results still oscillate the servo behavior a little bit on its starting point. And this is where tuning for the PID is needed.

PID tuning means auto-tune the manual calculation to precise the servo behavior while it changes its speed.

This tuning in the MATLAB needed the kP, kI, and kD values, followed by the sample time of the PID which then can be manipulated by its response time and transient behavior on a tuning graph such as shown in this Figure 2. 1. 4 below.

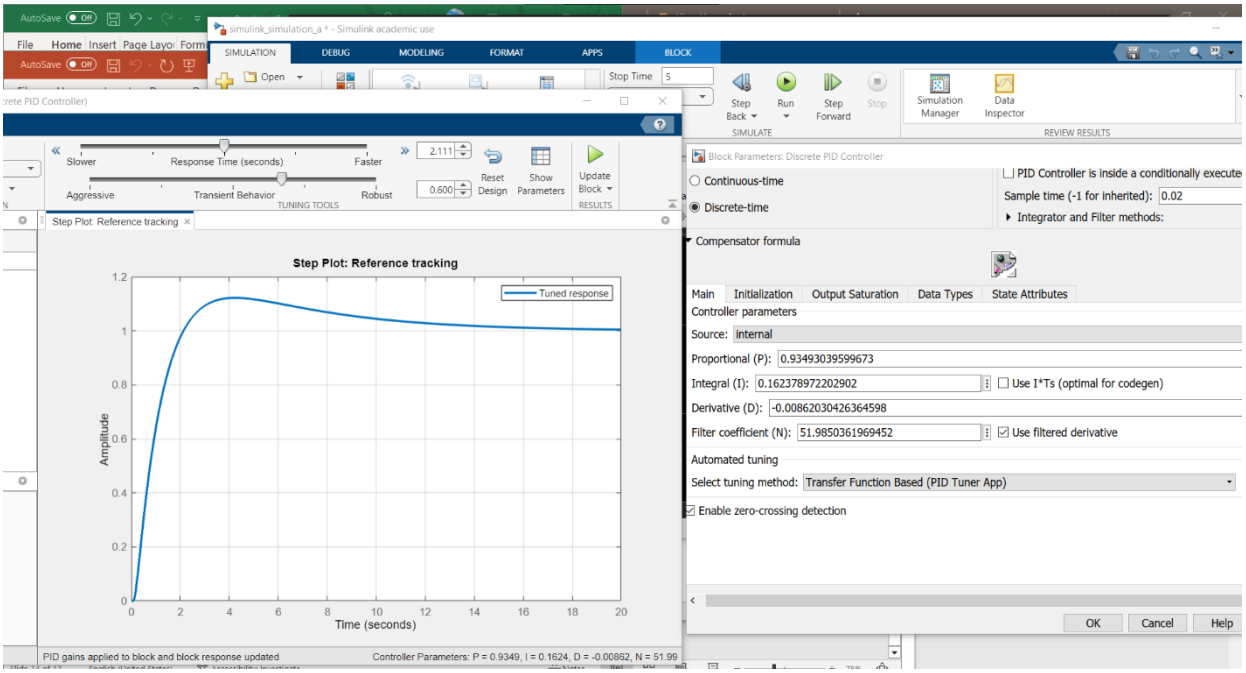

Figure 2. 1. 4 PID auto-tune result on MATLAB Simulink.

However, adjusting this tuning in Simulink needs time to make it correct. As can be seen in this image above for example, there is still a negative value resulted from the tuning while the graph itself still over bounce a little bit from 2 until 12 seconds which is still needed to be smoothen.

And by adjusting the response time and transient behavior properly and repeatedly, this smooth PID behavior such as shown in this Figure 2. 1. 5 below can be received.

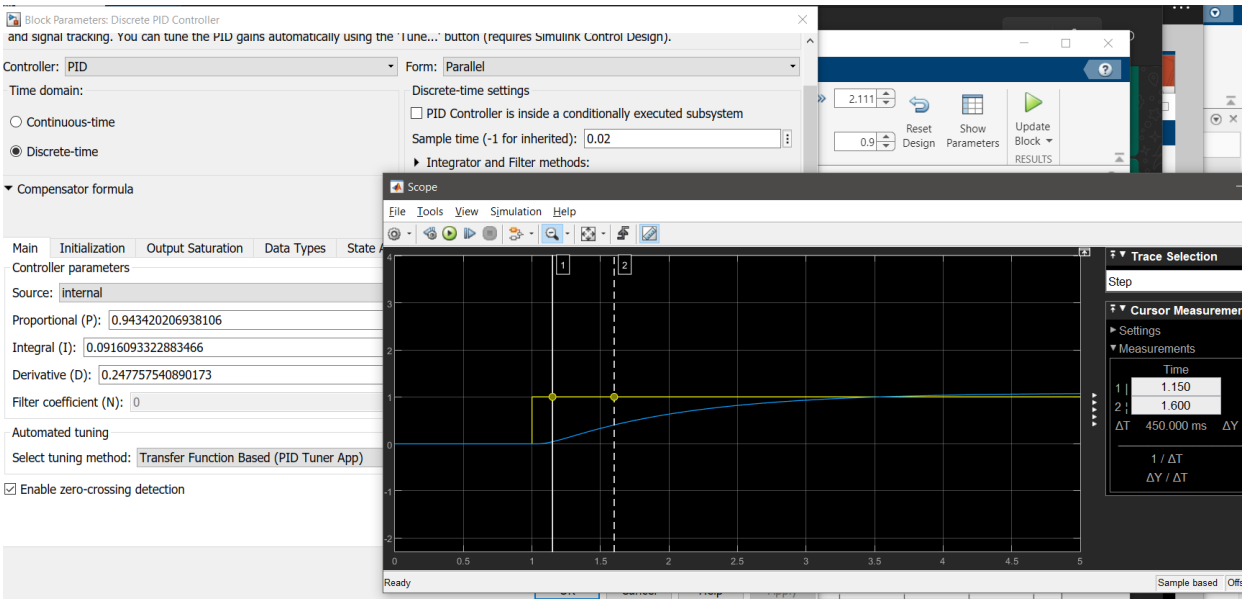

Figure 2. 1. 5 PID results after auto-tuning.

And from that tuning, it is decided that the PID data which will be used for the program are:

kP = 0.943420206938106

kI = 0.0916093322883466

kD = 0.247757540890173

2.2. Making The Pseudo Code

Making the pseudo code for the PID is rather much simpler than by doing the calculations and tuning for the PID in the MATLAB Simulink.

First, all calculated values from the kI, kP, and kD must be declared as a constant values in the code such as shown in this Figure 2. 2. 1 below.

```
47  /* USER CODE BEGIN PV */
48  const double kI = 0.549378309652703;
49  const double kP = 0.156755704961953;
50  const double kD = 0.0904541280843227;
51
52  double my_pid_values;
53  double measured_points;
54  /* USER CODE END PV */
```

Figure 2. 2. 1 kP, kI, kD constant variables.

After that, some functions that might be needed for the feedback controlling method must be declared. In this Figure 2. 2. 2 below, my_pid() is the function that will calculate the movement PID to the system setpoint. And the measure_distance() is a pseudo function that will be used by a certain hardware to detect the set point distance.

```
59  double my_pid(double, double);
60  double measure_distance(void);
```

Figure 2. 2. 2 functions declaration.

After declaring the functions, now is the time to work on the PID implementation. This Figure 2. 2. 3 below is the first step of the PID implementation, which is calculating the error value.

```
149 double my_pid(double obstacle_distance, double measured_distance) {
150 static double integral_val = 0;
151 static double prev_error = 0;
152
153 //1. Calculate error as the difference between the obstacle distance and the measured
154 //distance.
155 double error_val = measured_distance - obstacle_distance;
156
157 if (error_val == 0 || error_val > obstacle_distance) {
158     integral_val = 0;
159 }
```

Figure 2. 2. 3 error value measurements.

Error value is a distance estimation to reach the set point, based on the obstacle distance and the measured distance subtraction. The measured distance is a distance value from the measuring sensor tool. And the obstacle distance is the minimum distance that the system allowed to face before it hit something in front of it.

After the error value is measured, the PID can be calculated using the kP, kI, and kD as what is shown on this Figure 2. 2. 4 below. But to be mindful, the integral and previous error were declared as static in the previous Figure 2. 2. 3 above because the integral only increase and decrease per proportional value change. And the previous error always follows the current error value after the measurement and obstacle distance change. This previous error value will determine the derivative value by subtracting the current error value.

```
//2.a. calculate Proportional from error value and the kP
double proportional = error_val * kP;

//2.b. Integral from error value and the kI
integral_val += error_val;
integral_val = integral_val * kI;

//2.c. Derivative from error value's difference, then the kD
double derivative = error_val - prev_error;
derivative = derivative * kD;

prev_error = error_val;
```

Figure 2. 2. 4 PID calculations.

After the proportional, integral, and derivative values have found, the PID value can be resulted from the SUM of those three values. As what is shown in this Figure 2. 2. 5 below.

```
173 //3. Total PID output
174 double pid_output = proportional + derivative + integral_val;
```

Figure 2. 2. 5 Total PID calculation.

Then, an if statement to prevent a big oscillation like in this Figure 2. 2. 6 below can be added to the code, so the system can change the servo speed smoothly by using the proper PID values.

```
176 //4. Set PID result to prevent a big oscalation.
177 if (pid_output > MAX_OUTPUT_POWER) {
178     pid_output = MAX_OUTPUT_POWER;
179 }
180
181 return pid_output;
182 }
```

Figure 2. 2. 6 PID value restrictor for preventing the big oscillation value.

This calculation can be done in the main while loop of the robot program, such is shown in this Figure 2. 2. 7 below.

```
100     while (1) {
101         /* USER CODE END WHILE */
102
103         /* USER CODE BEGIN 3 */
104         measured_points = measure_distance();
105         my_pid_values = my_pid(OBSTACLE_POINT, measured_points);
106     }
107     /* USER CODE END 3 */
```

Figure 2. 2. 7 PID function implementation.

And the obstacle point, and the max output power can be written as constant values on the top of the program as a public constant variable, such as shown in this Figure 2. 2. 8 below.

```
34 /* USER CODE BEGIN PD */
35 #define MAX_OUTPUT_POWER 200
36 #define OBSTACLE_POINT 20
37
38 /* USER CODE END PD */
```

Figure 2. 2. 8 constant variables for set point and max power limit.

3. Recommendation

Personally, I really recommend using the MATLAB Simulink for the PID calculation because this app may reduce the oscillation of the system effectively.

Working on PID with only manual calculations will results many big errors value and very difficult to make the system smoothly change the speed.

Moreover, values from the MATLAB Simulink simulation are much more accurate than the manual measurement of each PID even if it uses the Ziegler-Nichols method.

4. References

Along this practicum, such documents and references are being used.

- Simulink Basic Tutorial (Fontys T3 Canvas)
- Introduction to PID controllers ed2 (George Gillard), November 10th 2017
- ParallaxStepResponse excel file

Git link to access the source file of the assignment:

- Timers output:

https://git.fhict.nl/I424848/t-3-pers-spring-24-farros-ramzy/-/tree/main/es/4-feedback-control-systems